



Aplicación de Diccionarios

Proyecto de prácticas · Curso de Java Enterprise Edition

Documentación del proyecto

Sesiones 1 a 9 · Versión V5

31 de julio de 2026

Proyecto de prácticas del curso de **Java Enterprise Edition**. Es un buscador de palabras que ha ido creciendo sesión a sesión: empezó siendo diez ficheros `.java` sueltos en una carpeta y hoy es un sistema distribuido, con su base de datos y con **tres interfaces de usuario** intercambiables.

Que una aplicación funcione se da por descontado. Si no funciona, no es una aplicación: es una colección de ficheros de texto con pretensiones. Lo que marca la diferencia es que **envejezca bien**: que un cambio grande en el sistema se traduzca en un impacto pequeño en el código que ya está escrito.

Ese es el hilo conductor de todo lo que hay aquí dentro.

Índice

1. [Qué hace](#)
 2. [Arranque rápido](#)
 3. [Por qué está partido en tantos trozos](#)
 4. [Arquitectura](#)
 5. [Los módulos](#)
 6. [Cómo se arranca cada pieza](#)
 7. [El API REST](#)
 8. [El modelo de datos](#)
 9. [La evolución del sistema](#)
 10. [Decisiones de diseño que conviene entender](#)
 11. [Limitaciones conocidas](#)
 12. [Para seguir leyendo](#)
-

1. Qué hace

Busca una palabra en un diccionario y devuelve sus significados. Si la palabra no existe, sugiere las más parecidas.

```
# Palabra que existe
c:\> buscarPalabra "casa" "es"
Aplicación de Diccionarios v1.1.0
La palabra casa existe en el diccionario de es, y significa:
- Edificio destinado a ser habitado.
- Familia o linaje.
Gracias por usar nuestra aplicación de diccionarios.

# Palabra que NO existe: el sistema sugiere alternativas (distancia de Levenshtein)
c:\> buscarPalabra "manana" "es"
Aplicación de Diccionarios v1.1.0
La palabra manana no existe en el diccionario de es.
Quizás quiso decir alguna de estas palabras:
- MAÑANA
Gracias por usar nuestra aplicación de diccionarios.

# Idioma que no tenemos
c:\> buscarPalabra "melón" "klingon"
Aplicación de Diccionarios v1.1.0
Lo siento, pero no tengo diccionario klingon.
Gracias por usar nuestra aplicación de diccionarios.

# Sin parámetros
c:\> buscarPalabra "melón"
Aplicación de Diccionarios v1.1.0
No ha suministrado los parámetros necesarios
Debe suministrar la palabra a buscar y el idioma del diccionario.
Ejemplo:
  c:\> buscarPalabra "melón" "es"
Gracias por usar nuestra aplicación de diccionarios.
```

Búsquedas insensibles a mayúsculas y minúsculas: `CASA`, `Casa` y `casa` dan el mismo resultado.

2. Arranque rápido

Requisitos: JDK 21 y Maven 3.9+. Si además vas a levantar la interfaz web, Node.js (probado con la 24.17) y npm.

```
# 1. Compilar e instalar todos los módulos (desde esta carpeta)
mvn clean install

# 2. Arrancar el servidor. Tarda ~1 minuto: carga 647.000 palabras en la BBDD.
#   Déjalo corriendo en su propia terminal.
mvn -pl servicio-web spring-boot:run

# 3. En OTRA terminal, arrancar el cliente que quieras:
mvn -pl aplicacion-completa exec:exec@acto2      # escritorio (JavaFX)
cd ui-web && npm install && npm start              # web (Angular, en :4200)
```

Para comprobar que el servidor responde, sin cliente ninguno:

```
curl http://localhost:8080/diccionarios          # ["EN","ELFICO","ES.GRANDE","ES"]
curl http://localhost:8080/diccionarios/es/casa
```

3. Por qué está partido en tantos trozos

No queremos una aplicación que sea un solo fichero: acabaría siendo enorme e imposible de mantener. Queremos una aplicación hecha de **componentes**, y que **cada componente tenga una única responsabilidad**.

Piensa en un coche. Tiene cientos de componentes —ruedas, alternador, batería, bujías— y cada uno hace una cosa y la hace bien. Eso importa por dos razones:

- Si se rompe un componente que hace una sola cosa, pierdes esa función, pero el resto del coche sigue andando.
- Puedes **reemplazar** cada pieza: las ruedas por desgaste, o por cambiar a neumáticos de invierno; el alternador porque se rompió, o porque has instalado un equipo de música que pide más potencia.

Y para poder reemplazarlas, existen los **estándares**. Una rueda se describe así: 17 pulgadas, 225 de ancho, perfil 45, código de velocidad V. Cualquier fabricante que cumpla esa especificación te vale.

Llevado a Java, esto son tres cosas distintas que conviene no confundir:

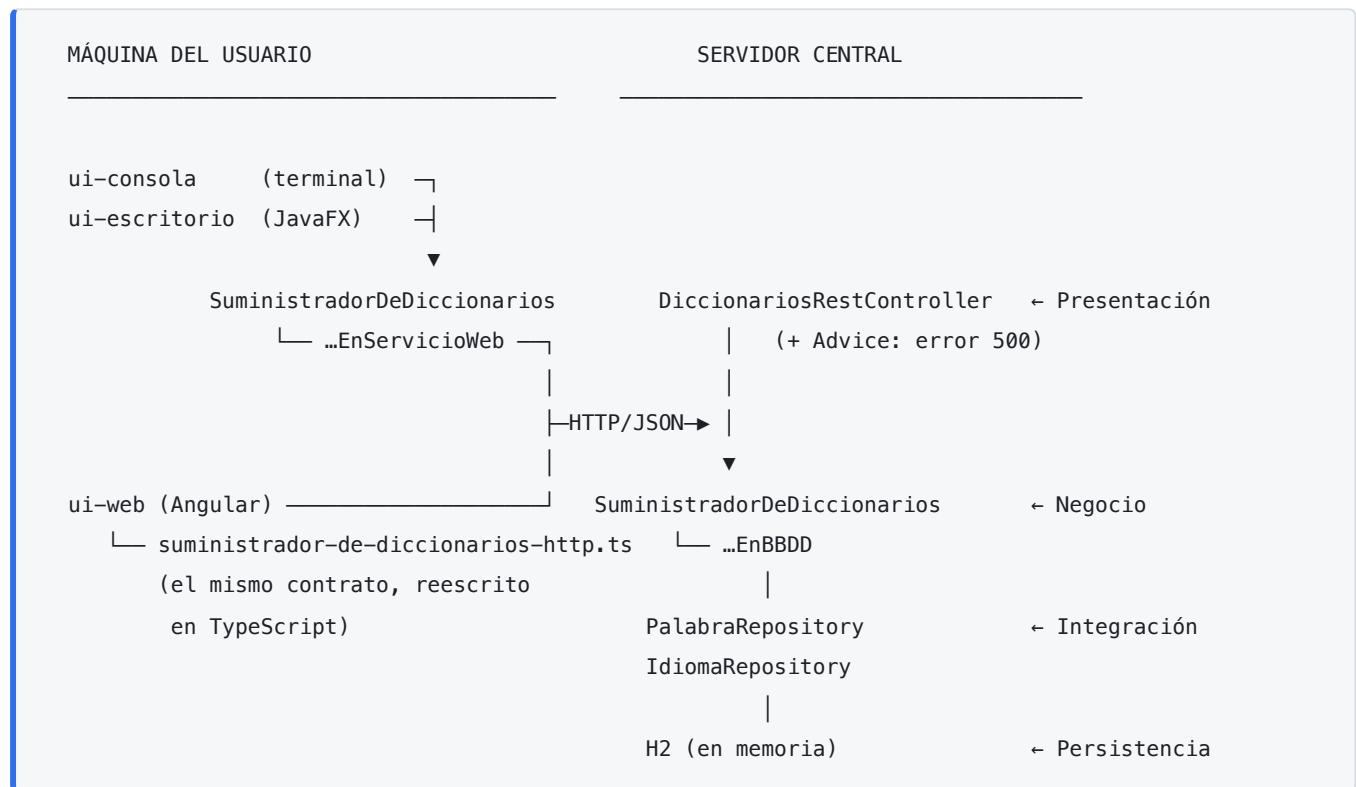
En el coche	En Java	Qué es
La especificación de la rueda	Interfaz	Algo abstracto: normas que hay que cumplir. No es una rueda.
El modelo <code>PIRELLI XB17J</code>	Clase	Un modelo concreto que cumple la especificación. Tampoco es una rueda.
La rueda que montas en el coche	Instancia	Esto sí. Lo tangible.

Por eso `Diccionario` y `SuministradorDeDiccionarios` son **interfaces**: describen lo que se puede hacer con un diccionario, no cómo se hace. Y por eso podemos tener diccionarios en ficheros, en base de datos o al otro lado de una red, y al resto del sistema le da igual cuál esté usando.

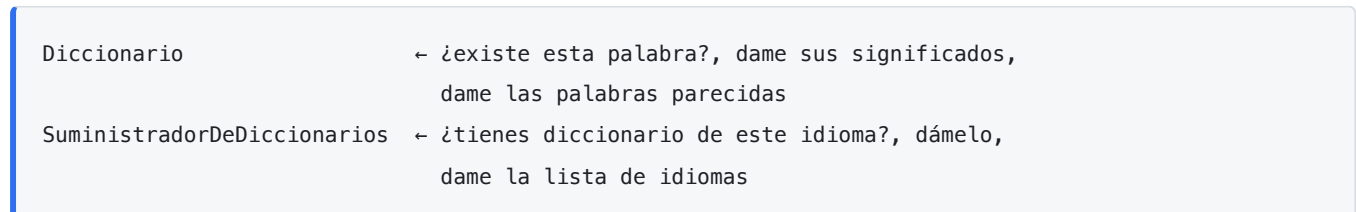
El desarrollo de la metáfora completa, tal y como se contó en la primera sesión, está en [README-original-dia1.md](#).

Y una nota sobre el oficio, hoy. El trabajo ya no es teclear código: eso lo escriben en buena medida los agentes de IA. El trabajo es entender qué componentes necesita el sistema, definir sus especificaciones y decidir cómo encajan. Después se le pide a la IA que escriba cada pieza, y se prueba y se integra. Este proyecto es la demostración: las interfaces de usuario web y de escritorio se construyeron así, y encajaron porque las abstracciones ya estaban bien puestas.

4. Arquitectura



Las dos abstracciones sobre las que se sostiene todo viven en `diccionarios-api`:



Fíjate en que `SuministradorDeDiccionarios` aparece a **los dos lados** del dibujo. En el caso de la consola y el escritorio es literalmente el mismo módulo Java, reutilizado en cliente y servidor. La aplicación web, al vivir fuera de Java, tuvo que reescribir esa capa en TypeScript... y llegó exactamente al mismo contrato. Esa es la ventaja de haber separado la abstracción de sus implementaciones.

5. Los módulos

Proyecto Maven multimódulo: un `pom.xml` padre sin código que agrupa doce módulos. Todos en la versión `1.0.0`, salvo `diccionario-elfico`, que va por la `1.1.0` porque se le añadieron palabras: **cada módulo se versiona por su cuenta**, que es justo para lo que sirve el versionado semántico.

Contratos (sin implementación)

Módulo	Qué define
<code>diccionarios-api</code>	<code>Diccionario</code> , <code>SuministradorDeDiccionarios</code> y el cálculo de la distancia de Levenshtein
<code>ui-api</code>	<code>InterfazDeUsuario</code>

Datos

Módulo	Contenido
<code>diccionario-es</code>	Español: <code>es.txt</code> (331 palabras) y <code>es.grande.txt</code> (646.612)
<code>diccionario-en</code>	Inglés: 383 palabras
<code>diccionario-elfico</code>	Élfico: 4 palabras

Formato de los ficheros, una línea por palabra:

```
palabra=significado 1|significado 2|significado 3
abanico=Herramienta que sirve para mover el aire.|Conjunto de opciones entre las que elegir.
```

Implementaciones de dónde salen los diccionarios

Módulo	Origen de los datos
<code>diccionarios-en-ficheros</code>	Ficheros <code>.txt</code> del <code>classpath</code>
<code>diccionarios-en-bbdd</code>	Base de datos relacional, vía JPA/Hibernate
<code>diccionarios-en-servicio-web</code>	Llamadas HTTP a un servidor REST

Interfaces de usuario

Módulo	Tecnología	Contrato que implementa
<code>ui-consola</code>	Terminal	<code>InterfazDeUsuario</code>
<code>ui-escritorio</code>	JavaFX 21	<code>InterfazDeUsuario</code> (Acto 1) y ventana autónoma (Acto 2)
<code>ui-web</code>	Angular 22	Ninguno: consume <code>SuministradorDeDiccionarios</code> en TypeScript

Ejecutables

Módulo	Qué es
<code>servicio-web</code>	El servidor: Spring Boot 4.1, controlador REST y gestión de errores
<code>aplicacion-completa</code>	El cliente: <code>main</code> , factorías y orquestación

`ui-web` es un proyecto npm independiente y **no forma parte del build de Maven**: se compila y se arranca con sus propias herramientas.

6. Cómo se arranca cada pieza

El servidor

```
mvn -pl servicio-web spring-boot:run
```

Levanta un Tomcat embebido en el puerto 8080 y una base de datos H2 **en memoria**. Al arrancar, `CargadorDeDatos` vuelca los ficheros `.txt` en la base de datos, lo que tarda alrededor de un minuto por las 647.000 palabras. Como la base de datos vive en memoria, **cada arranque parte de cero**, que es justo lo que interesa para probar.

El cliente de consola

```
mvn -pl aplicacion-completa exec:java -Dexec.args="casa es"
```

⚠ **Ojo con el estado actual.** La factoría `InterfazDeUsuarioFactory` está configurada para devolver la interfaz **de escritorio**, así que ese comando abre una ventana en lugar de escribir en la terminal. Para volver a consola hay que comentar una línea y descomentar la otra en `InterfazDeUsuarioFactory.java`. Que cambiar la interfaz de usuario de todo el sistema sea eso —una línea— es precisamente lo que se quería demostrar.

El cliente de escritorio

```
mvn -pl aplicacion-completa exec:exec@acto1    # implementa InterfazDeUsuario tal cual
mvn -pl aplicacion-completa exec:exec@acto2    # aplicación autónoma (la buena)
```

Se usa `exec:exec` y no `exec:java` porque las ventanas gráficas en macOS exigen el hilo principal del proceso, y `exec:java` ejecuta el código dentro del proceso de Maven en un hilo secundario. `exec:exec` lanza una JVM nueva y limpia.

El **Acto 1** implementa el contrato `InterfazDeUsuario` al pie de la letra, y sirve para ver que la sustitución funciona... y también para ver que el resultado no es usable: sólo permite una búsqueda, porque el contrato está pensado para un guion que se ejecuta una vez. El **Acto 2** es la aplicación de escritorio de verdad: consume directamente `SuministradorDeDiccionarios`, con selector de idioma, sugerencias pulsables y tema claro/oscuro.

El cliente web

```
cd ui-web
npm install    # sólo la primera vez
npm start     # http://localhost:4200
```

En desarrollo, `proxy.conf.json` redirige las peticiones a `/diccionarios` hacia `http://localhost:8080`, de forma que el navegador cree que todo viene del mismo sitio. Es lo que evita tener que configurar CORS

mientras se desarrolla.

7. El API REST

Este contrato se diseñó **antes** de escribir el código del servidor, para poder entregárselo al equipo que iba a construir el cliente. Es el equivalente moderno del IDL de CORBA; hoy se escribiría formalmente en OpenAPI.

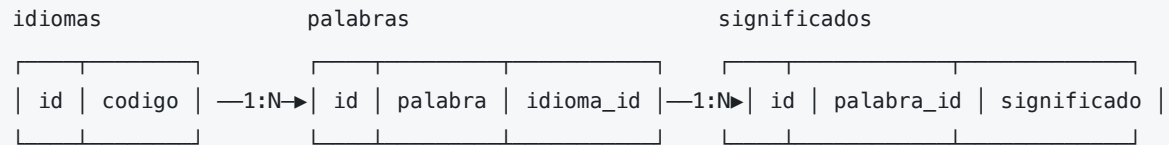
Petición	Situación	Estado	Cuerpo
GET /diccionarios/test	Prueba de vida	200	Hola desde DiccionariosRestController
GET /diccionarios	Siempre	200	["EN","ELFICO","ES.GRANDE","ES"]
GET /diccionarios/{idioma}	El diccionario existe	200	(vacío)
GET /diccionarios/{idioma}	No existe	404	(vacío)
GET /diccionarios/{idioma}/{palabra}	Ambos existen	200	{"idioma":"es","palabra":"casa","significados":[...],"similares":null}
GET /diccionarios/{idioma}/{palabra}	El idioma existe, la palabra no	404	{"idioma":"es","palabra":null,"significados":null,"similares":["MAÑANA"]}
GET /diccionarios/{idioma}/{palabra}	El idioma no existe	404	{"idioma":null,"palabra":null,"significados":null,"similares":null}
cualquiera	Error no controlado en el servidor	500	El mensaje de la excepción

Los cuerpos de la tabla son los que **realmente** devuelve el servidor hoy, no los que se diseñaron. Hay dos diferencias con el contrato de la sesión 4, y conviene conocerlas:

- Se documentó que un idioma inexistente devolvería {}, pero Jackson serializa también los campos a null, así que llegan los cuatro.
- El campo idioma se devuelve **tal y como lo escribió el cliente** (es), no normalizado a mayúsculas (ES) como se guarda en la base de datos.

El 500 no está escrito en el controlador. Lo produce DiccionariosRestControllerAdvice, una clase anotada con @RestControllerAdvice que envuelve al controlador sin tocarlo. En el controlador sólo se escribe el happy path.

8. El modelo de datos



Restricciones de integridad:

- `codigo` es único en `idiomas`.
- La combinación (`palabra`, `idioma_id`) es única: la misma palabra puede existir en varios idiomas, pero no dos veces en el mismo.
- La combinación (`significado`, `palabra_id`) es única: un significado no se puede repetir dentro de la misma palabra.

Estas tablas **no se crean a mano**: Hibernate las genera a partir de las anotaciones `@Entity`, `@ManyToOne` y `@OneToMany` de las clases del paquete `entidades`. Los códigos de idioma y las palabras se guardan **en mayúsculas**, que es como se resolvió que las búsquedas no dependan de mayúsculas y minúsculas.

9. La evolución del sistema

Versión	Sesión	Qué cambió
V1	1–3	Aplicación de consola, monolítica, diccionarios en ficheros <code>.txt</code>
V2	4–5	Se parte en módulos Maven y se separa en cliente y servidor REST
V3	6	La persistencia pasa a base de datos con JPA e Hibernate
V4	7–8	Listado de idiomas, búsquedas sin distinguir mayúsculas, errores 500 con AOP y sugerencias por distancia de Levenshtein con <i>streams</i>
V5	9	Dos interfaces de usuario nuevas: web (Angular) y escritorio (JavaFX)

Lo interesante es el coste de cada salto:

- De V2 a V3 (de ficheros a base de datos): un módulo nuevo y **una dependencia cambiada** en un `pom.xml`.
- De V4 a V5 (interfaz de escritorio): un módulo nuevo y **una línea** modificada en la factoría. En todo el sistema se borraron dos líneas, y una era una línea en blanco.

Se reutilizaron sin tocarlos `diccionarios-api`, `ui-api`, `ui-consola`, los tres módulos de diccionarios y el cliente HTTP.

10. Decisiones de diseño que conviene entender

Factoría o inyección de dependencias, según el lado. En el cliente, quién construye cada componente lo decide una factoría escrita a mano (`SuministradorDeDiccionariosFactory`). En el servidor eso lo hace el contenedor de Spring con `@Component` . Son la misma idea; la segunda te la regala el framework. La versión con `@Configuration` y `@Bean` se conserva comentada en `SuministradorDeDiccionariosConfiguration` para poder compararlas.

Dos clases `RespuestaPalabra` , una en el servidor y otra en el cliente. Parece código duplicado y no lo es: una cosa es lo que el servidor decide enviar y otra lo que el cliente necesita recibir. Si el servidor añade un campo, el cliente no tiene por qué enterarse. Hoy son iguales; mañana no tienen por qué serlo, y eso está bien. La aplicación de Angular hace lo mismo separando el DTO del modelo de dominio.

Métodos `default` en las interfaces. Cuando en la sesión 7 se añadió `dameIdiomas()` a `SuministradorDeDiccionarios` , tres módulos dejaban de compilar. La solución fue declararlo como `default` , para que sólo lo implementara quien lo necesitaba. Dos sesiones después, la aplicación de escritorio necesitó ese método en el cliente HTTP: añadirlo costó un método y cero cambios en el resto. Es el principio Abierto/Cerrado cobrado con intereses.

El contrato `InterfazDeUsuario` no encaja con una interfaz gráfica. Está escrito para que la aplicación mande sobre la interfaz ("*dame la palabra*", "*muestra esto*"), que es como funciona un programa de terminal. En una aplicación gráfica manda el usuario, y el flujo se invierte. Por eso ni la web ni el escritorio (en su versión buena) lo implementan: las dos consumen directamente `SuministradorDeDiccionarios` . **De las dos abstracciones del diseño original, la que ha sobrevivido a tres clientes es la de negocio, no la de interfaz de usuario.**

Programación funcional para las palabras parecidas. Recorrer 647.000 palabras calculando distancias se resuelve con un `stream` de siete operaciones (`map` → `filter` → `map` → `filter` → `sorted` → `map` → `limit`) en `DiccionarioEnBBDD.palabrasSimilares()` . En programación imperativa serían páginas de código, más difíciles de leer y más lentas.

11. Limitaciones conocidas

Cosas que hoy no están bien resueltas. Están aquí a propósito: sirven de material para las siguientes sesiones.

- **ES.GRANDE es un idioma fantasma.** El código de idioma se deduce del nombre del fichero, así que `es.grande.txt` se carga como un idioma distinto de `ES`. Se cuela en el desplegable de las interfaces gráficas. Una convención de nombres de fichero se ha convertido en contrato de datos sin que nadie lo decidiera.
 - **El cliente de consola hace tres viajes por la red** para resolver una sola búsqueda, porque `tienesDiccionarioDe`, `existe` y `palabrasSimilares` abren cada uno su propia petición. Cuando la palabra no existe, el servidor calcula las distancias de Levenshtein **dos veces**. Las interfaces web y de escritorio lo resuelven con una o dos llamadas.
 - `palabrasSimilares()` **se trae el diccionario entero a memoria** en cada fallo de búsqueda. Con H2 en memoria se aguanta; contra una base de datos real, no.
 - **H2 1.4.200 es de 2019** y arrastra vulnerabilidades conocidas. Para clase da igual, pero no se subiría así a producción.
 - **CORS está sin resolver.** En desarrollo lo tapa el proxy de Angular; el día que la web se sirva desde un origen distinto al del API, habrá que configurarlo.
 - **La versión que imprime la aplicación (v1.1.0) no coincide** con la de los `pom.xml` (1.0.0), porque está escrita a mano en el código.
 - **Elegir interfaz de usuario obliga a recompilar**, porque la decisión está en la factoría. Podría ser un parámetro de arranque o una propiedad de configuración.
-

12. Para seguir leyendo

Documento	Contenido
<code>../notas/temario-y-equivalencias.md</code>	El temario oficial punto por punto: qué se dio, dónde está el ejemplo, y qué tecnologías han quedado obsoletas (RMI, CORBA, JNDI, SOAP) y por qué
<code>../notas/dia1.md</code> ... <code>dia9</code>	Las notas de cada sesión
<code>README-original-dia1.md</code>	La versión original de este documento: el diseño del sistema tal y como se planteó en la primera sesión, antes de escribir una sola línea
<code>../diccionarios/</code>	La versión 1 del proyecto, conservada sin tocar como punto de comparación: diez ficheros sueltos, sin módulos ni paquetes

El código está comentado a conciencia, con la explicación de por qué se tomó cada decisión. Merece más la pena leerlo que leer sobre él.